

C Files and Streams

Based on ECM2433 L08 files and streams: standard streams, file modes, buffering, errno, perror, feof, fseek, and rewind.

Description: What You Need To Know

1. Streams and standard streams

A stream is an input/output abstraction. The standard streams are stdin, stdout, and stderr. Streams may be text or binary and are usually buffered.

2. Opening and closing files

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    return 1;
}

/* use fp */

fclose(fp);
```

Common modes include r, w, and a for text files, and rb, wb, and ab for binary files.

3. Formatted console input

scanf returns the number of items successfully assigned. For numeric input, pass the address of the variable so scanf can write into it.

```
int k;
if (scanf("%d", &k) != 1) {
    fprintf(stderr, "bad integer\n");
    return 1;
}
```

4. Stream reading and writing

- fgetc reads one character.
- fputc writes one character.
- fgets reads a line or up to a size limit.
- fputs writes a string without its null terminator.
- fprintf writes formatted output to a stream.
- fscanf reads formatted input from a stream.

5. Buffering and flushing

Output may be held in a buffer before it appears. Use fflush on an output stream when the buffered data must be written immediately.

6. Errors and file positions

Inspect errno only after an actual failure. After read loops, use perror to detect an error and feof to detect end-of-file. Use rewind and fseek to control the file position.

```
rewind(fp);
fseek(fp, 20, SEEK_SET);
```

Practice Questions

- 1 Explain the difference between a text stream and a binary stream.
- 2 Write code to open numbers.txt for reading and handle failure safely.
- 3 State what scanf returns and why numeric input needs the variable address.
- 4 Name three stream input/output functions and say what each does.
- 5 Explain why output from printf or fprintf may not appear immediately.
- 6 State when it is valid to inspect errno.
- 7 Explain the difference between perror(fp) and feof(fp).
- 8 Write the calls to rewind a file and then move to byte offset 20 from the beginning.

- 9 Explain why `while ( !feof(fp) )` is usually a bad file-reading loop pattern.
- 10 Write a short fragment that opens a file, writes one line, flushes it, and closes it.

Mark Scheme

Use this after attempting the questions.

- 1 Text streams contain characters and may perform translations such as newline handling. Binary streams contain bytes with no translation.
- 2 Expected pattern: `FILE *fp = fopen("numbers.txt", "r");`, then check `fp == NULL` before use.
- 3 It returns the number of successful assignments. Numeric input uses the variable address because `scanf` must write into the caller's object.
- 4 Examples: `fgetc` reads a character, `fgets` reads a line up to a limit, `fprintf` writes formatted output to a stream.
- 5 Streams are buffered. `fflush` forces buffered output to be written.
- 6 Only inspect `errno` after a call has failed, such as `fopen` returning `NULL`. A successful call does not necessarily clear an old `errno`.
- 7 `ferror` reports a stream error. `feof` reports that end-of-file has been reached after a read attempt.
- 8 Expected: `rewind(fp); fseek(fp, 20, SEEK_SET);`.
- 9 EOF is only known after a read attempt fails. Check the read function result directly instead.
- 10 Open with `fopen`, check for `NULL`, call `fprintf`, call `fflush` if immediate output matters, then `fclose`.

Useful Pattern

```
#include <errno.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *fp = fopen("out.txt", "w");
    if (fp == NULL) {
        fprintf(stderr, "open failed: %s\n", strerror(errno));
        return 1;
    }

    fprintf(fp, "Value = %d\n", 42);
    fflush(fp);
    fclose(fp);
    return 0;
}
```